

闽江御盾：AI AGENT 驱动的动态攻防推演靶场

部署手册

一、靶场底层安装

1.1 部署架构

闽江御盾靶场的部署架构融合了先进的硬件、网络及软件技术，以实现高效、安全且灵活的网络安全演练环境。该架构采用分层设计理念，各层级既独立运行又协同配合，为攻防推演提供稳定可靠的基础支撑。

1.1.1 硬件层架构

硬件层是靶场运行的物理基础，通过高性能设备的合理搭配，满足靶场对计算、存储、日志分析等核心需求。

主控节点选用高性能服务器，配备 Intel Xeon Platinum 8380 CPU，拥有 64 核心 128 线程，基础频率 2.3GHz，睿频可达 3.2GHz，128GB DDR4 ECC 内存（频率 3200MHz），具备强大的计算与调度能力，负责整个靶场的核心管理与控制，包括资源分配、任务调度、节点监控等，确保系统稳定运行。

计算节点采用多台戴尔 PowerEdge R750 服务器，每台搭载两颗 AMD EPYC 7763 CPU（共 128 核心 256 线程，基础频率 2.45GHz，睿频 3.5GHz），配备 256GB DDR4 内存（频率 2933MHz），为各类靶机（如模拟企业服务器、物联网设备等）的运行提供充足的计算资源，支持多场景并行演练。

存储节点基于华为 OceanStor 18500F 存储阵列，配置 20 块 15K RPM SAS 1TB 硬盘组成 RAID 50 阵列，兼具高速读写与数据冗余能力，可存储靶机镜像、演练数据、用户配置等信息，保障数据的安全与快速访问。

日志服务器采用联想 ThinkSystem SR650，配备 Intel Xeon Silver 4316 CPU（16 核心 32 线程，基础频率 2.3GHz，睿频 2.8GHz），64GB DDR4 内存（频率 2666MHz），专门用于收集、存储和分析靶场运行过程中产生的海量日志信息（如攻击行为日志、系统操作日志等），为演练复盘与安全分析提供数据支持。

节点类	服务器型号 / 硬件平台	CPU 参数	内存参数	存储配置	主要功能描述

型					
主控节点	高性能服务器	Intel Xeon Platinum 8380, 64 核心 128 线程, 基础频率 2.3GHz, 睿频 3.2GHz	128GB DDR4 ECC, 3200MHz	-	负责靶场核心管理与控制, 包括资源分配、任务调度、节点监控等, 确保系统稳定运行。
计算节点	戴尔 PowerEdge R750	2 颗 AMD EPYC 7763, 共 128 核心 256 线程, 基础频率 2.45GHz, 睿频 3.5GHz	256GB DDR4, 2933MHz	-	为各类靶机 (模拟企业服务器、物联网设备等) 提供计算资源, 支持多场景并行演练。
存储节点	华为 OceanStor 18500F 存储阵列	-	-	20 块 15K RPM SAS 1TB 硬盘, 组成 RAID 50 阵列	存储靶机镜像、演练数据、用户配置等信息, 兼具高速读写与数据冗余能力。
日志服务器	联想 ThinkSystem SR650	Intel Xeon Silver 4316, 16 核心 32 线程, 基础频率 2.3GHz, 睿频 2.8GHz	64GB DDR4, 2666MHz	-	收集、存储和分析靶场运行日志 (攻击行为日志、系统操作日志等), 支持演练复盘。

1.1.2 网络层硬件架构

网络层硬件负责靶场内部及内外交互的数据传输, 通过高性能设备与安全策略, 保障网络通畅与安全隔离。

核心交换机选用华为 CloudEngine 16800-48C, 提供 48 个 100GE 端口, 具备高达 12.8Tbps 的背板带宽, 支持海量数据高速转发, 是靶场内部各节点 (主控、计算、存储等) 通信的核心枢纽。

搭配深信服 AD 系列负载均衡器, 通过智能算法 (如轮询、加权最少连接等) 将用户请求与内部业务流量均匀分配至各节点, 避免单点过载, 提升系统整体性能与可用性。

部署网御星云防火墙, 具备万兆级别的吞吐能力, 支持入侵检测与防御 (IDS/IPS)、防病毒、URL 过滤等功能, 严格控制网络访问 (如限制外部对靶机的非授权访问、隔离不同演练场景的网络), 保障靶场网络安全。

1.1.3 软件层架构

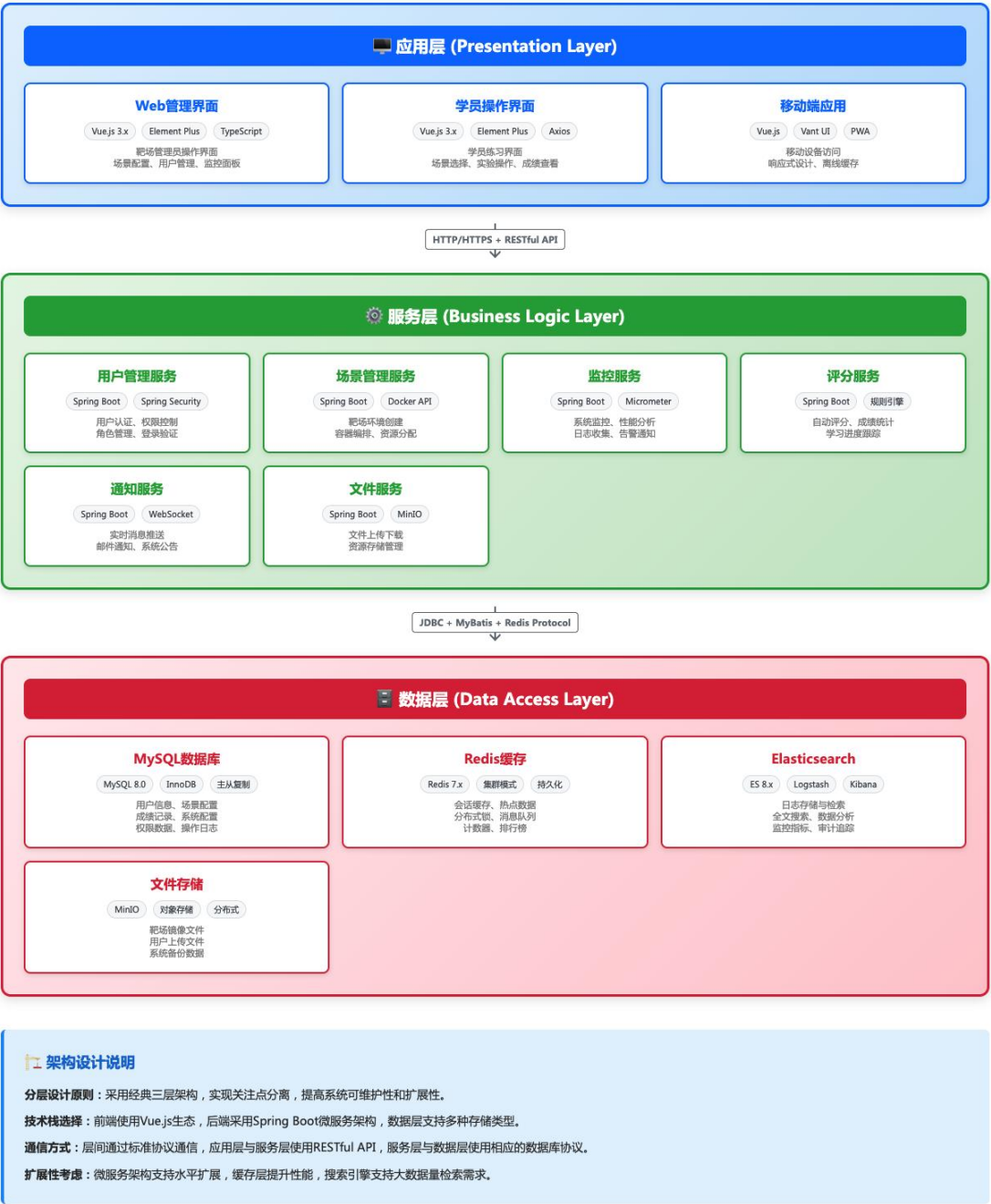
软件层架构基于分层设计, 实现用户交互、业务逻辑处理与数据存储的解耦, 提升系统扩展性与维护性。

应用层主要面向用户，提供直观友好的 Web 界面，采用 Vue.js 框架结合 Element UI 组件库开发，具备响应式设计，可适配不同终端设备（PC、平板等），方便用户进行场景创建、演练操作、结果查看等。

服务层基于 Spring Cloud 微服务架构，将系统拆分为多个独立服务，如用户管理服务（负责账号、权限管理）、场景管理服务（负责演练场景的创建与生命周期管理）、攻击防御服务（负责调度攻击/防御 AGENT）等，各服务间通过 RESTful API 进行通信，实现高内聚、低耦合，便于单独扩展与维护。

数据层使用多类型数据库协同存储：MySQL 8.0 关系型数据库存储结构化数据（如用户信息、演练配置、权限规则等）；Elasticsearch 7.17 作为分布式搜索引擎，存储和检索非结构化日志数据（如攻击行为日志、系统运行日志），支持全文检索与实时分析；Redis 6.2 作为缓存数据库，存储热点数据（如用户会话、高频访问的场景配置），减少数据库访问压力，提升系统响应速度。

闽江御盾靶场软件层三层架构图



1.2 底层系统安装

靶场底层系统以开源技术为核心，结合容器化与 AI 能力，构建灵活可扩展的基础运行环境，包括云计算平台、容器管理平台、大语言模型及 AGENT 组件的部署。

1.2.1 Openstack 开源云计算平台安装

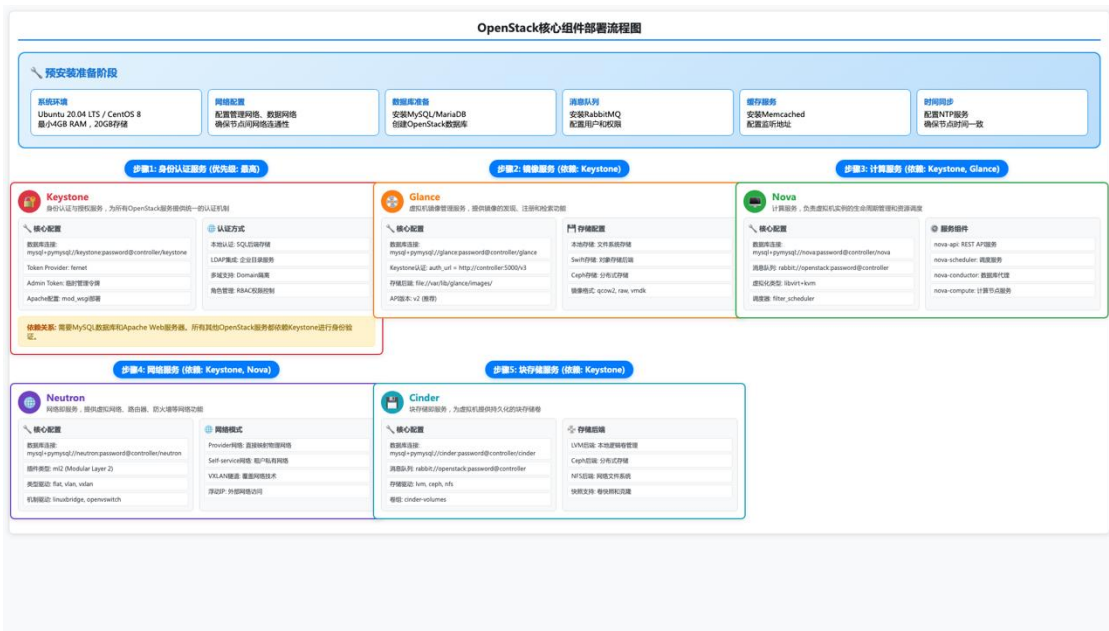
Openstack 作为靶场的云计算基础，提供虚拟机生命周期管理、网络配置、镜像管理等核心能力，支撑靶机的动态创建与资源分配。

安装前需确保服务器硬件满足要求（如 CPU 支持虚拟化技术、内存 $\geq$ 64GB、硬盘 $\geq$ 500GB），并配置好网络（区管理网、业务网等）。以 Ubuntu 20.04 作为操作系统基础，先安装相关依赖包：OpenStack 客户端工具

（python3-openstackclient）、数据库服务（MySQL 8.0 或 MariaDB 10.5）、消息队列服务（RabbitMQ）、缓存服务（Memcached）等。

接着按照 OpenStack 官方文档步骤，依次安装和配置各个核心组件：Keystone（身份认证服务，负责用户与服务的身份验证）、Glance（镜像服务，管理靶机操作系统镜像）、Nova（计算服务，调度虚拟机创建与运行）、Neutron（网络服务，配置虚拟机网络与子网）、Cinder（块存储服务，为虚拟机提供持久化存储）等。

配置过程中，需根据实际需求设置各组件参数，如网络拓扑（选择 Flat、VLAN 等网络模式）、存储配置（本地存储或共享存储）、认证方式（令牌有效期、加密算法）等，通过 OpenStack CLI 或 Dashboard 验证各组件间通信正常，最终构



建出稳定的云平台。

1.2.2 Kubernetes 容器管理平台安装

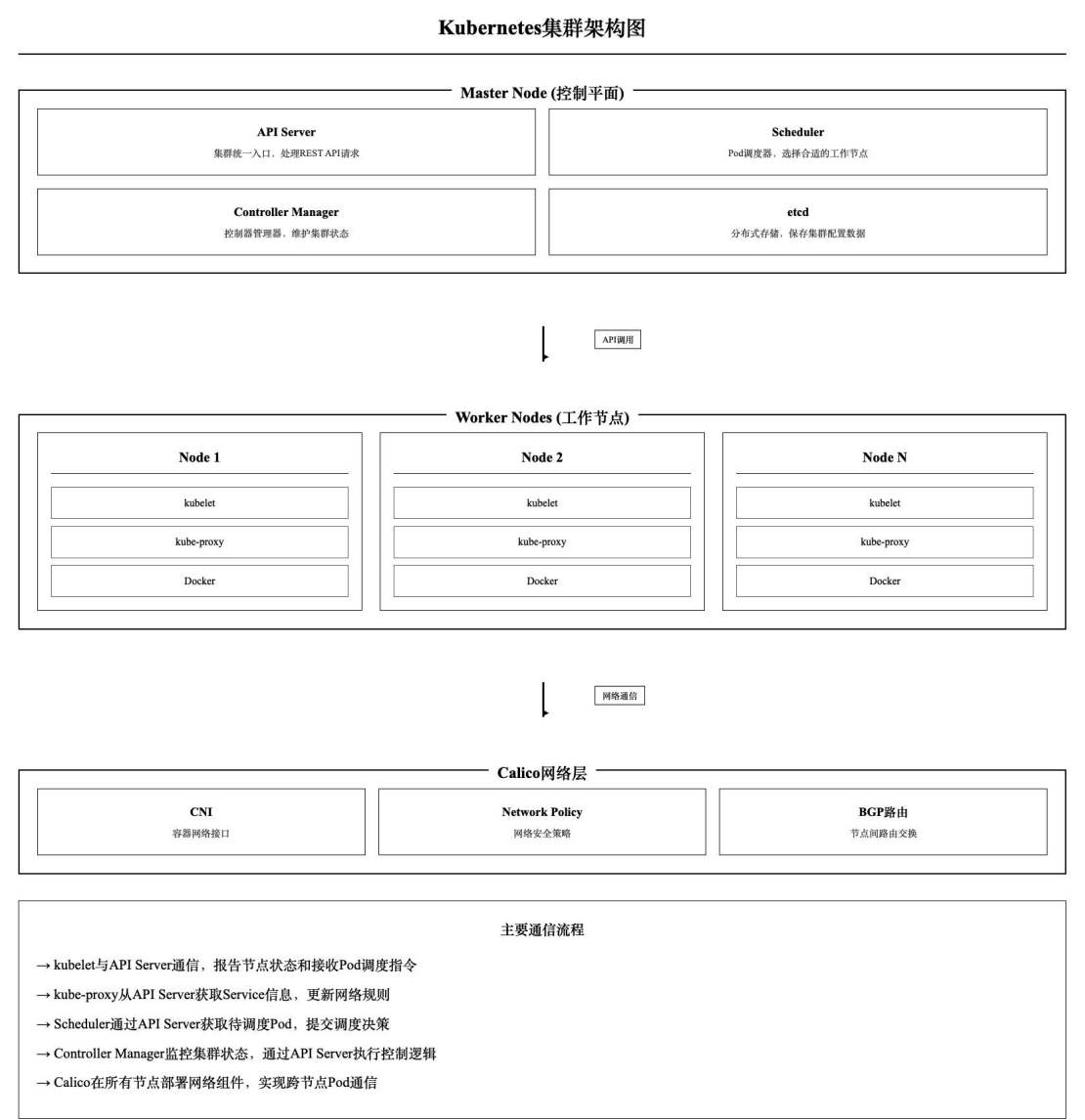
Kubernetes (K8S) 负责对 Docker 类型靶机容器进行编排与管理，支持容器的快速部署、扩展与自愈，提升靶场资源利用率。

采用 kubeadm 工具进行快速部署：首先在各节点（主节点与工作节点）上安装 Docker 容器运行时（版本 $\geq$ 20.10），配置 Docker 镜像加速器以提升镜像拉取

速度；接着安装 kubeadm、kubect、kubectl 工具（版本需保持一致，建议选用 1.24+稳定版）。

初始化 Kubernetes 主节点：通过 kubeadm init --pod-network-cidr=10.244.0.0/16 命令完成主节点的初始化配置（--pod-network-cidr 指定容器网络网段），生成相关配置文件和证书；执行提示命令配置 kubectl 客户端（如复制 admin.conf 到 ~/.kube/config）。

加入工作节点：在工作节点上执行主节点初始化完成后输出的 kubeadm join 命令（包含令牌与证书哈希），将工作节点加入到 Kubernetes 集群；安装完成后，部署 Calico 网络插件（kubectl apply -f https://docs.projectcalico.org/v3.24/manifests/calico.yaml），为容器提供高效的网络通信能力，实现容器间的互联互通。



1.2.3 本地大语言模型（Qwen-32B）部署

本地大语言模型 Qwen-32B 为靶场提供智能问答、漏洞分析等本地化语言处理能力，无需依赖外部网络，保障数据安全性。

部署前需确保服务器配备 NVIDIA A100 GPU 加速卡（至少 1 张，建议 2 张以上以提升推理速度），安装 GPU 驱动（版本 $\geq$ 470.57.02）、CUDA 11.7+、cuDNN 8.4+ 等深度学习运行环境，通过 nvidia-smi 命令验证 GPU 与驱动是否正常工作。

从官方仓库（如阿里云灵积模型服务）下载 Qwen-32B 模型权重文件（需遵守模型使用协议），放置于 /data/models/qwen-32b 目录；使用 Hugging Face 的 Transformers 库加载模型，示例代码如下：

```
from transformers import AutoTokenizer, AutoModelForCausalLM

tokenizer = AutoTokenizer.from_pretrained("/data/models/qwen-32b")
model = AutoModelForCausalLM.from_pretrained(
    "/data/models/qwen-32b",
    device_map="auto",
    torch_dtype="auto"
)
# 配置超参数
model.config.max_new_tokens = 1024 # 最大生成 tokens 数
model.config.temperature = 0.7 # 生成多样性（0-1，值越小越确定）
```

启动模型服务后，可通过内部 API 接口为靶场提供漏洞描述生成、攻击步骤解释等功能，根据服务器硬件资源调整批处理大小（建议初始设为 2-4）、学习率等超参数，平衡性能与效率。

1.2.4 线上大语言模型（Deepseek）接入

线上大语言模型 Deepseek 具备卓越的复杂问题处理能力，通过 API 接入为靶场补充智能攻击策略生成、防御建议提供等高级功能。

首先在 Deepseek 官方平台注册账号，申请 API 密钥（API Key），并了解接口规范（如请求 URL、参数格式、返回数据结构）；Deepseek 采用先进的混合专家架构（MoE）和多头潜在注意力机制（MLA），支持多轮对话与结构化输出，适合处理攻防场景中的复杂逻辑推理。

在靶场系统中编写调用代码，示例如下：

```
import requests
import json
```

```

url = "https://api.deepseek.com/v1/chat/completions"
headers = {
    "Content-Type": "application/json",
    "Authorization": "Bearer your_api_key"
}
data = {
    "model": "deepseek-chat",
    "messages": [
        {"role": "system", "content": "你是靶场攻防策略分析师，需基于漏洞类型生成攻击步骤"},
        {"role": "user", "content": "针对 SQL 注入漏洞，生成详细攻击步骤"}
    ]
}
response = requests.post(url, headers=headers, data=json.dumps(data))
result = response.json()
print(result["choices"][0]["message"]["content"])

```

根据实际应用场景（如攻击策略生成、防御建议）对接口返回的数据进行二次处理，添加错误重试机制（如网络超时重连），确保调用稳定性。

1.2.5 AGENT 开发与部署

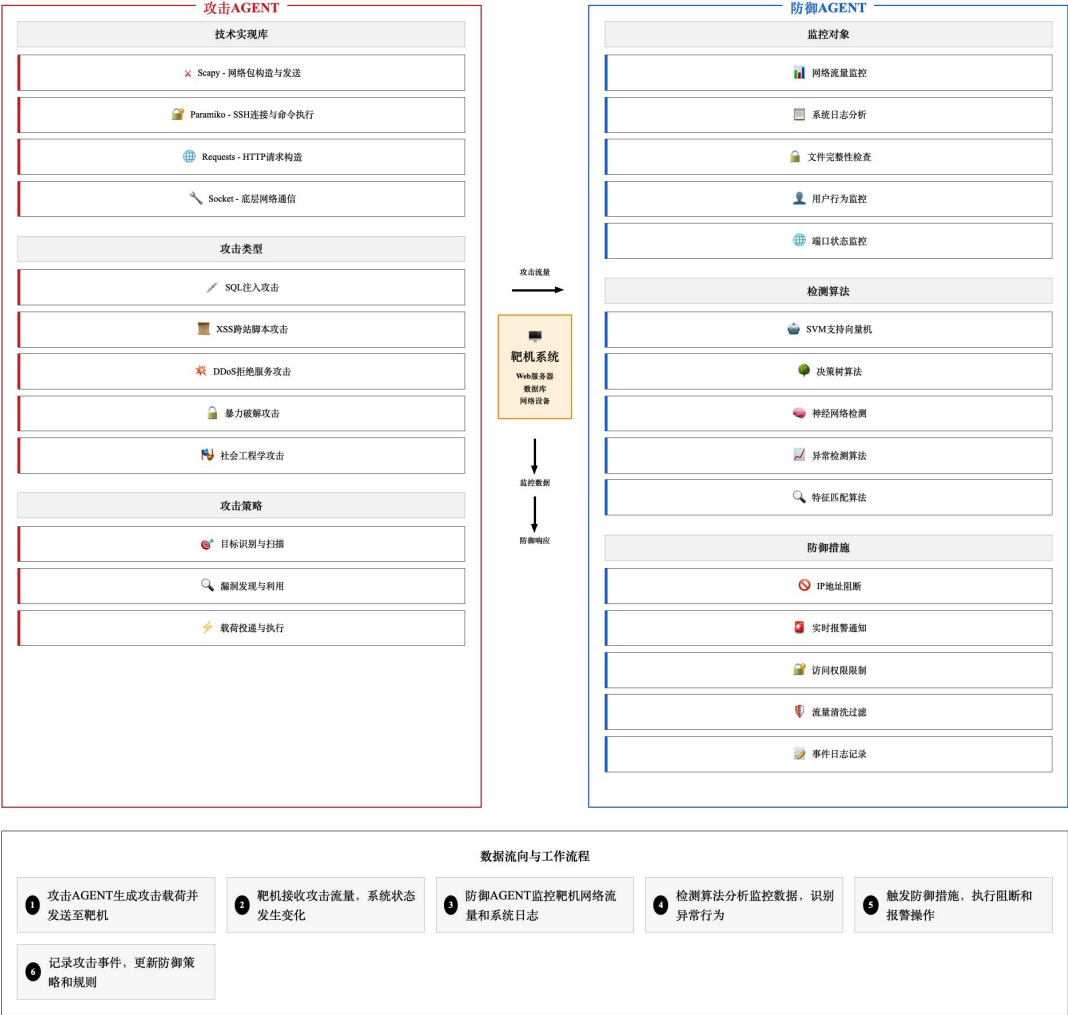
AGENT 是靶场实现动态攻防推演的核心组件，分为攻击 AGENT 与防御 AGENT，通过 Python 语言结合 Flask 框架开发，实现自动化攻击与实时防御。

攻击 AGENT 利用 Python 丰富的库资源实现多种攻击技术：Scapy 库用于构造与发送网络数据包（如模拟 DDoS 攻击）、Paramiko 库实现 SSH 远程连接（用于弱口令爆破）、Sqlmap-api 库集成 SQL 注入检测与利用功能、BeautifulSoup 库解析网页内容（辅助 XSS 攻击）等。攻击 AGENT 可模拟真实攻击场景（如 SQL 注入获取数据库信息、XSS 窃取 cookie），并记录攻击过程与结果。

防御 AGENT 实时监控靶机的网络流量（通过 tcpdump 抓包分析）、系统日志（/var/log/目录下的 auth.log、syslog 等）、进程状态（通过 ps 命令）等信息，利用机器学习算法（如支持向量机 SVM、决策树）进行异常检测。例如，通过分析 HTTP 请求特征识别 SQL 注入攻击，一旦发现异常，立即采取阻断（调用防火墙 API 添加拦截规则）、报警（发送邮件或短信至管理员）等防御措施。

将 AGENT 打包为 Docker 镜像，通过 Kubernetes Deployment 部署在集群中，配置资源限制（如 CPU≤2 核、内存≤4GB）与自动扩缩容规则（如根据靶机数量动态调整 AGENT 实例数），实现快速部署、扩展与维护。

攻击AGENT与防御AGENT工作流程图



二、靶场软件安装

在完成靶场底层安装后，需进行靶场软件的安装与配置，实现用户交互与自动化集成功能，包括浏览器访问与API接口两种方式。

2.1 浏览器访问配置

浏览器访问是靶场最直观的使用方式，用户通过Web界面完成场景创建、演练管理等操作，需进行访问地址配置与证书处理。

2.1.1 访问地址与端口配置

靶场Web服务默认部署在主控节点，端口为8080（可通过配置文件修改，如/etc/minjiangyudun/web.conf中的server_port参数）。若靶场部署在内网环

境，访问地址为服务器内网 IP+端口，如“http://192.168.1.100:8080”；若通过公网访问，需配置域名解析（将域名指向服务器公网 IP），并开放防火墙 8080 端口（或通过 Nginx 反向代理至 443 端口），访问地址为“https://minjiangyudun.com”。

2.1.2 证书处理与登录操作

首次访问时，由于靶场默认使用自签名证书（用于测试环境），浏览器会提示“安全风险”（如 Chrome 显示“您的连接不是私密连接”）。用户需确认访问地址正确后，通过浏览器高级选项选择“继续访问”（不同浏览器步骤略有差异：Chrome 点击“高级”→“继续前往 192.168.1.100（不安全）”）。

进入登录页面后，输入管理员预先分配的账号（如 admin）和密码（初始密码通常为 Admin@123，建议首次登录后修改），点击“登录”按钮进入 Web 管理界面。

2.1.3 Web 界面核心功能操作

Web 管理界面包含多个功能模块，核心操作如下：

场景创建：点击左侧菜单“场景管理”→“场景创建”，在弹出的对话框中选择场景模板（如“金融支付系统安全演练”“企业办公网络安全测试”），设置参数（靶机数量：1-20 台；操作系统类型：Windows Server 2019、Ubuntu 20.04 等；漏洞类型：SQL 注入、XSS 攻击、缓冲区溢出等），点击“确定”后，系统自动在 K8S 集群中创建靶机实例。

演练监控：在“演练管理”→“实时监控”页面，可查看靶机运行状态（CPU/内存使用率、网络流量）、攻击 AGENT 的攻击记录（攻击时间、类型、结果）、防御 AGENT 的防御动作（阻断次数、报警信息），支持通过时间轴回放演练过程。

用户管理：管理员在“系统设置”→“用户管理”中，可创建用户账号、分配角色（管理员、普通用户、访客）、设置权限（如是否允许创建场景、查看日志），确保操作安全性。

2.2 API 接口配置与使用

API 接口为有自动化需求或系统集成需求的用户提供访问方式，支持场景管理、攻击防御等功能的程序化调用。

2.2.1 API 密钥获取与权限配置

使用 API 前需获取 API 密钥：登录 Web 管理界面，点击右上角用户名→“个人设置”→“API 密钥管理”，点击“生成密钥”按钮，系统生成一串 32 位唯一密钥（如“sk_8a3b7c9d0e1f2g3h4i5j6k7l8m9n0o”），点击“复制”保存密钥（密钥仅显示一次，丢失需重新生成）。

管理员可在“系统设置”→“API 权限管理”中，为不同用户的 API 密钥配置权限（如仅允许创建场景、禁止发起攻击），限制 API 调用范围，提升安全性。

2.2.2 场景创建 API 调用示例

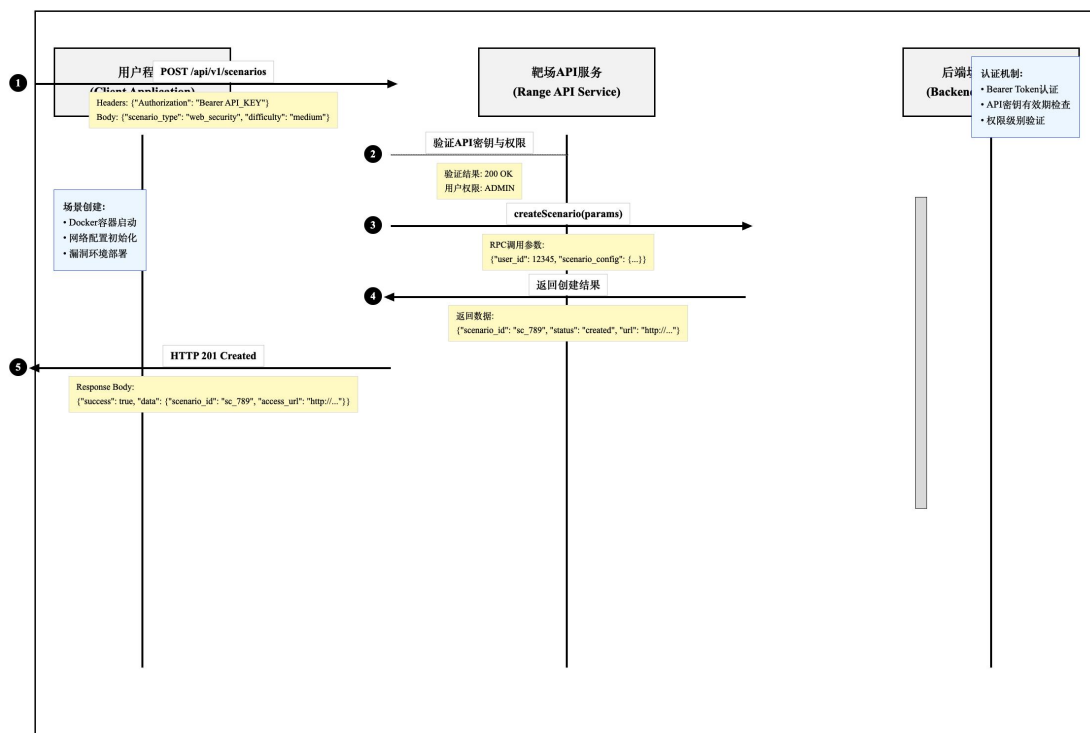
以创建“金融支付系统安全演练”场景为例，使用 Python 的 requests 库调用 API，代码如下：

```
import requests
import json

url = "http://192.168.1.100:8080/api/scenes"
headers = {
    "Authorization": "Bearer sk_8a3b7c9d0e1f2g3h4i5j6k7l8m9n0o", # 替换为实际 API 密钥
    "Content-Type": "application/json"
}
data = {
    "template_name": "金融支付系统安全演练",
    "target_number": 10, # 靶机数量
    "os_type": ["Windows Server 2019", "Ubuntu 20.04"], # 操作系统类型
    "vulnerability_type": ["SQL 注入", "XSS 攻击", "弱口令"] # 漏洞类型
}
response = requests.post(url, headers=headers, data=json.dumps(data))
if response.status_code == 200:
    print("场景创建成功，场景 ID: ", response.json()["scene_id"])
else:
    print("场景创建失败，错误信息: ", response.text)
```

接口返回 200 表示成功，返回场景 ID（如“scene_123456”）可用于后续查询场景状态；常见错误码：401（密钥无效）、400（参数错误，如靶机数量超过上限）、500（服务器内部错误）。

API调用时序图



2.2.3 其他常用 API 接口说明

除场景创建外，靶场还提供以下常用 API 接口：

- 场景状态查询**：GET `/api/scenes/{scene_id}`，返回场景当前状态（运行中、已结束、异常）及靶机列表。
- 发起攻击任务**：POST `/api/attacks`，参数包括场景 ID、攻击类型、目标靶机 IP，返回攻击任务 ID。
- 获取演练日志**：GET `/api/logs?scene_id={scene_id}`，返回指定场景的攻击/防御日志（JSON 格式）。

用户可参考靶场提供的 API 文档（访问 <http://192.168.1.100:8080/api/docs> 查看），实现更复杂的自动化操作。

三、靶场使用方法

闽江御盾靶场支持浏览器手动操作与 API 自动化调用两种使用方式，满足不同用户的需求（如教学演示、自动化测试），以下详细介绍具体操作流程。

3.1 浏览器访问使用方法

通过浏览器访问靶场适合手动创建场景、实时监控演练过程，操作步骤简单直观，适合初学者或需要可视化操作的场景。

3.1.1 访问前准备与地址输入

使用前需确保客户端设备与靶场服务器网络连通（可通过 ping 192.168.1.100 测试），并安装主流浏览器（Chrome 90+、Firefox 88+、Edge 90+）。

打开浏览器，在地址栏准确输入靶场访问地址：内网环境输入“http://服务器内网 IP:端口”（如 http://192.168.1.100:8080）；公网环境输入域名（如 https://minjiangyudun.com），按回车键发起访问。

3.1.2 证书警告处理与登录

若靶场使用自签名证书（非生产环境常见），浏览器会弹出安全警告，此时需确认访问地址为靶场服务器（避免钓鱼网站），然后按以下步骤处理：

Chrome 浏览器：点击警告页面中的“高级”→“继续前往 192.168.1.100（不安全）”；Firefox 浏览器：点击“高级”→“接受风险并继续”；Edge 浏览器：点击“详细信息”→“继续访问”。

进入登录页面后，输入账号（如 user01）和密码（如 User@123456），点击“登录”按钮：若账号密码正确，进入 Web 管理界面；若错误，提示“用户名或密码错误”，需重新输入。

3.1.3 核心功能使用详解

场景创建与启动：

1. 在左侧菜单点击“场景管理”→“场景列表”，点击“+ 新建场景”按钮；
2. 在弹出的“场景配置”窗口，选择模板（如“物联网设备安全测试”）；
3. 设置参数：靶机数量设为 5 台，操作系统选择“Ubuntu 20.04(物联网版)”，漏洞类型勾选“弱口令”“命令注入”；
4. 点击“下一步”，设置演练时长（如 2 小时）、是否启用自动防御（勾选“是”）；
5. 点击“创建并启动”，系统开始部署靶机，页面显示“部署中（1/5）”，完成后状态变为“运行中”。

演练监控与操作：

1. 进入“演练管理”→“当前演练”，选择目标场景，点击“监控”；
2. 在监控页面，左侧为靶机列表（显示 IP、漏洞状态），右侧为实时日志（如“攻击 AGENT 对 192.168.2.10 发起弱口令爆破，成功”）；

3. 点击某台靶机的“详情”，可查看其 CPU 使用率、网络连接、进程列表等信息；
4. 若需手动干预，点击“防御操作”→“添加防火墙规则”，输入攻击源 IP（如 192.168.3.5），设置“阻断”，点击“确认”生效。

结果分析与报告导出：

1. 演练结束后，点击“场景管理”→“历史场景”，选择目标场景，点击“生成报告”；
2. 报告包含攻击成功率、防御有效率、漏洞分布统计等数据，支持图表展示（如漏洞类型饼图）；
3. 点击“导出报告”，可选择 PDF 或 Excel 格式，保存至本地用于分析或分享。

3.2 API 接口使用方法

API 接口适合需要自动化集成的场景（如批量创建场景、与 CI/CD 流程结合），通过编程实现靶场操作，提升效率。

3.2.1 API 调用基础配置

在编程环境中（如 Python、Java），需配置 API 基础参数：访问地址（如 `http://192.168.1.100:8080/api`）、API 密钥（放入请求头 `Authorization` 字段，格式为“`Bearer 密钥值`”）、请求格式（JSON）。

以 Python 为例，基础调用框架如下：

```
import requests
import json

BASE_URL = "http://192.168.1.100:8080/api"
API_KEY = "sk_8a3b7c9d0e1f2g3h4i5j6k7l8m9n0o"

def api_request(method, path, data=None):
    url = f"{BASE_URL}{path}"
    headers = {
        "Authorization": f"Bearer {API_KEY}",
        "Content-Type": "application/json"
    }
    if method == "GET":
        response = requests.get(url, headers=headers, params=data)
    elif method == "POST":
        response = requests.post(url, headers=headers,
                                data=json.dumps(data))
```

```
return response
```

3.2.2 自动化场景管理示例

以下代码实现批量创建 3 个不同场景，并查询其状态：

```
# 批量创建场景
```

```
scenes = [  
    {  
        "template_name": "企业办公网络安全演练",  
        "target_number": 5,  
        "vulnerability_type": ["SQL 注入", "XSS 攻击"]  
    },  
    {  
        "template_name": "工业控制系统演练",  
        "target_number": 3,  
        "vulnerability_type": ["缓冲区溢出", "协议漏洞"]  
    },  
    {  
        "template_name": "移动应用安全测试",  
        "target_number": 4,  
        "vulnerability_type": ["API 漏洞", "本地存储泄露"]  
    }  
]  
  
scene_ids = []  
for scene in scenes:  
    response = api_request("POST", "/scenes", scene)  
    if response.status_code == 200:  
        scene_ids.append(response.json()["scene_id"])  
        print(f"场景创建成功: {scene['template_name']}, ID:  
{scene_ids[-1]}")  
    else:  
        print(f"场景创建失败: {scene['template_name']}, 错误:  
{response.text}")  
  
# 查询场景状态  
for scene_id in scene_ids:  
    response = api_request("GET", f"/scenes/{scene_id}")  
    if response.status_code == 200:  
        status = response.json()["status"]  
        print(f"场景 {scene_id} 状态: {status}")
```

3.2.3 攻击防御自动化示例

以下代码实现对指定场景发起攻击，并根据攻击结果动态调整防御策略：

```
# 发起攻击任务
def start_attack(scene_id, target_ip, attack_type):
    data = {
        "scene_id": scene_id,
        "target_ip": target_ip,
        "attack_type": attack_type,
        "attack_params": {"timeout": 30} # 攻击超时时间 30 秒
    }
    response = api_request("POST", "/attacks", data)
    return response.json() if response.status_code == 200 else None

# 调整防御策略
def adjust_defense(scene_id, attack_source_ip):
    data = {
        "scene_id": scene_id,
        "action": "block", # 阻断动作
        "source_ip": attack_source_ip,
        "duration": 3600 # 阻断 1 小时（秒）
    }
    response = api_request("POST", "/defenses/strategy", data)
    return response.status_code == 200

# 执行流程
scene_id = "scene_123456"
target_ip = "192.168.2.10"
attack_result = start_attack(scene_id, target_ip, "SQL 注入")

if attack_result and attack_result["success"]:
    print(f"攻击成功，攻击源 IP: {attack_result['source_ip']}")
    # 阻断攻击源
    if adjust_defense(scene_id, attack_result["source_ip"]):
        print("防御策略调整成功，已阻断攻击源")
    else:
        print("防御策略调整失败")
else:
    print("攻击失败或超时")
```

四、部署注意事项与常见问题解决

靶场部署过程涉及多类技术组件，需注意硬件兼容性、网络配置、安全策略等关键环节，以下总结常见问题及解决方法。

4.1 Openstack 部署注意事项与问题解决

4.1.1 部署前注意事项

1. 硬件兼容性：提前通过 OpenStack 官方硬件兼容性列表（HCL）检查服务器型号、网卡、存储设备是否支持，避免使用小众硬件导致驱动问题。
2. 网络规划：明确管理网（用于组件通信）、租户网（用于靶机网络）、外部网（用于外部访问）的 IP 段划分，禁用防火墙（如 ufw、firewalld）或开放必要端口（如 Keystone 的 5000 端口、Nova 的 8774 端口）。
3. 时间同步：所有节点通过 NTP 服务（如 chrony）同步时间，避免因时间偏差导致组件认证失败（执行 `timedatectl set-ntp yes` 启用时间同步）。

4.1.2 常见问题及解决

问题 1：Nova 服务启动失败，日志显示“无法连接到数据库”。

解决：检查 `/etc/nova/nova.conf` 中 `[database]` 部分的连接参数（如 `connection=mysql+pymysql://nova:nova_db_password@controller/nova`），确保 MySQL 服务正常运行（`systemctl status mysql`），且数据库用户权限正确（执行 `GRANT ALL PRIVILEGES ON nova.* TO 'nova'@'localhost' IDENTIFIED BY 'nova_db_password';`）。

问题 2：Neutron 网络无法创建，提示“没有可用的物理网络”。

解决：检查 `/etc/neutron/plugins/ml2/ml2_conf.ini` 中 `[ml2_type_flat]` 的 `flat_networks` 参数（如 `flat_networks=provider`），确保与 `/etc/neutron/plugins/ml2/linuxbridge_agent.ini` 中 `[linux_bridge]` 的 `physical_interface_mappings` 参数（如 `provider:ens33`）一致，且物理网卡 `ens33` 存在（`ip addr show` 验证）。

4.2 K8S 部署注意事项与问题解决

4.2.1 部署前注意事项

1. 系统一致性：所有节点使用相同操作系统版本（如 Ubuntu 20.04），内核版本 ≥ 5.4 ，禁用 Swap 分区（`swapoff -a`，并在 `/etc/fstab` 中注释 Swap 条目）。

2. 网络配置：确保节点间网络互通（尤其是 6443 端口，K8S API 通信），关闭 SELinux（setenforce 0），配置内核参数（如 net.bridge.bridge-nf-call-iptables=1）。

3. 镜像源：由于 K8S 镜像默认拉取自国外仓库，建议配置国内镜像源（如阿里云），执行 `kubeadm config images pull --image-repository registry.aliyuncs.com/google_containers` 预拉取镜像。

4.2.2 常见问题及解决

问题 1：工作节点执行 `kubeadm join` 失败，提示 “connection refused”。

解决：检查主节点 API Server 是否正常运行（`kubectl get pods -n kube-system`，确保 `kube-apiserver-xxx` 状态为 Running）；确认主节点 6443 端口开放（`netstat -tlnp | grep 6443`）；检查工作节点与主节点的网络连通性（`telnet 主节点 IP 6443`）。

问题 2：Pod 状态为 Pending，事件显示 “no nodes available to schedule pods”。

解决：检查节点是否被标记为不可调度（`kubectl get nodes`，STATUS 应为 Ready）；若节点状态为 NotReady，查看 kubelet 日志（`journalctl -u kubelet`），常见原因是容器运行时（Docker）未启动（`systemctl start docker`）或网络插件未部署（需先部署 Calico 等插件）。

4.3 大模型部署与配置注意事项

4.3.1 本地模型（Qwen-32B）部署注意事项

1. 硬件要求：Qwen-32B 模型推理需至少 16GB GPU 显存（推荐单卡 32GB 或多卡分布式推理），确保 GPU 驱动与 CUDA 版本兼容（如 GPU 驱动 470.57.02 对应 CUDA 11.7）。

2. 模型文件：从官方渠道下载模型权重，校验文件哈希值（如 SHA256）避免文件损坏；模型文件较大（约 60GB），需确保存储节点有足够空间，下载时使用断点续传工具（如 `wget -c`）。

3. 性能优化：推理速度过慢时，可启用模型量化（如 INT8 量化，需 Transformers 库支持），或调整 `batch_size`（建议 1-2）减少显存占用；通过 `nvidia-smi` 监控 GPU 利用率，若低于 50%，可增加并发请求数。

4.3.2 线上模型（Deepseek）配置注意事项

1. API 密钥安全：避免在代码中硬编码 API 密钥，建议使用环境变量（如 `export DEEPSEEK_API_KEY="your_key"`）或配置文件（权限设为 600）存储；定期轮换密钥（在官方平台操作）。
2. 调用限制：了解 API 的调用频率限制（如每分钟 60 次），避免触发限流；添加请求重试机制（如使用 `tenacity` 库），处理临时网络错误。
3. 数据处理：向线上模型发送的数据避免包含敏感信息（如靶场内部 IP、用户密码），如需发送日志，先脱敏处理（替换敏感字段为***）。

4.4 网络与安全配置注意事项

4.4.1 网络拓扑与防火墙配置

1. 网络隔离：将靶场网络划分为管理区（主控节点、存储节点）、靶机区（演练用靶机）、外部访问区，通过 VLAN 或防火墙规则限制区域间通信（如禁止靶机区直接访问管理区）。
2. 端口开放原则：仅开放必要端口（如 Web 服务 8080、SSH 22），关闭所有非必要端口（如数据库 3306 仅允许管理区访问）；防火墙规则遵循“白名单”机制，仅允许指定 IP 访问。
3. 网络监控：部署网络流量监控工具（如 Zabbix、Prometheus+Grafana），实时监控带宽使用、异常连接（如大量来自同一 IP 的连接请求），及时发现网络攻击。

4.4.2 系统安全配置

1. 系统补丁：定期更新操作系统与软件组件的安全补丁（Ubuntu 使用 `apt update && apt upgrade -y`，CentOS 使用 `yum update -y`），修复已知漏洞。
2. 用户权限：遵循最小权限原则，普通用户禁用 `sudo` 权限；管理员账号使用强密码（包含大小写字母、数字、特殊字符，长度 ≥ 12 位），定期更换（如每 90 天）。
3. 数据加密：敏感数据（如用户密码、API 密钥）存储时采用 AES-256 加密；数据传输使用 SSL/TLS（配置 Let's Encrypt 免费证书，替换自签名证书）。
4. 日志审计：开启系统日志（`/var/log/secure` 记录登录日志）、应用日志（Openstack/K8S 组件日志），通过 Elasticsearch+Kibana 分析日志，检测异常操作（如多次失败的 SSH 登录）。